

Worker LNB — Estado Técnico Completo

Prueba Vertical Pagaduría Digital · CONTRACT_PAYMENT_COMMITTED_V0.1 · 6 de octubre de 2026

Worker de la Pagaduría Digital — PoC vertical CONTRACT_PAYMENT_COMMITTED_V0.1 (proyecto LNB)

Worker (módulo de Cloud Run) de la prueba vertical de **LNB / Pagaduría Digital**: recibe el push de Pub/Sub de un pago confirmado (`PAYMENT_COMMITTED`), lo valida, lo hace pasar por el **Gobernador** (decisión de negocio) y el **Traductor** (plan SQL de 12 parámetros), persiste en un mock de **Sybase DES** con idempotencia atómica por `operationId` , cuarentena (DLQ) y recuperación in-doubt, y reporta a un endpoint (Opción B, mockeado). Estado de la entrega: `mvn test` **75/75 en verde** (+ harness E2E de 16 escenarios, 16/16 PASS) y demo en vivo 7/7 HTTP 200.

Nota sobre el alcance: Este README documenta el estado de la **PoC local** con mocks deterministas. La integración real con DEV (Vertex AI, Pub/Sub real, JDBC/Sybase) se documenta en `roadmap-agentes-vertex.md` (FASE 0-6). Cada fase del plan congelado (`lnb-docs/PLAN_IMPLEMENTACION_V0.1.md`) queda reflejada en `README.md` .

Cómo lo verá una persona del proyecto LNB: este repo es la pieza "adentro" del flujo `Outbox → Pub/Sub → Worker → Sybase DES` . Todo lo que no es el worker en sí (contrato de Alex, catálogo, casos de la prueba, plan, reporte de evidencia, fixture SQL) está duplicado en `lnb-docs/` para que el repo sea autocontenido.

1. Contenido del repo

```

worker-poc/
├─ pom.xml # Spring Boot 4.1.1, Java 21, Maven
├─ mvnw / mvnw.cmd # wrapper Maven (sin instalación necesari
a)
├─ README.md # este documento
├─ DEMO.md # runbook corto de la demo (tests + live)
├─ lnb-docs/ # esenciales del proyecto para repo autoco
ntenido
├─ └─ CONTRACT_EVENTO_V0.1.md # contrato oficial PAYMENT_COMMITTED d
e Alex (base)
├─ └─ CATALOG_SYNTHETIC_V0.json # whitelist sintética de referencia V0
├─ └─ PRUEBA_VERTICAL_CASOS_SYNTHETIC_V0.md # los casos que reproduce el har
ness
├─ └─ PLAN_IMPLEMENTACION_V0.1.md # PLAN CONGELADO (fases 1-6, decision
es, anexos)
├─ └─ PLAN_POC_JAVA_V1.md # plan de la PoC (fases, decisiones, a
nexos)
├─ └─ MINUTA_ALEX_RESULTADO_V1.md # minuta de Alex (base del contrato o
ficial)
├─ └─ AMBIENTE_DEV_LNB.md # ambiente/seguridad del entorno DEV
├─ └─ REPORTE_EVIDENCIA_POC_LNB.md # reporte con índices a la evidencia
├─ └─ FIXTURE_SYNTHETIC_DEV.sql # DDL sintético de las 2 tablas (DEV)
├─ docs/evidencia/ # snapshots congelados por fecha (2026-09-
15 V0 sintética, 2026-09-22 entrega actual)
├─ scripts/ # demo en vivo (arranque/jar, POSTs, stop
por :8080)
├─ └─ demo-live-start.ps1
├─ └─ demo-live-run.ps1
├─ └─ demo-live-stop.ps1
├─ src/
├─ └─ main/java/com/pagaduriasintetica/worker/ # código del worker (ver §3)
├─ └─ main/resources/application.properties # config trivial (solo nombr
e de app)
├─ └─ main/resources/catalog/CATALOG_PAYMENT_COMMITTED_V0.1.json # catálogo
en el classpath
├─ └─ test/java/com/pagaduriasintetica/worker/ # suites + harness de eviden
cia (ver §6)

```

target/ **NO se sube al repo** (.gitignore): es regenerable con `mvn test` y contiene el jar, la evidencia en vivo y una copia fresca de la misma.

2. Contexto de negocio y arquitectura

El proyecto LNB definió para la Pagaduría Digital una transición de **Sybase legacy hacia un**

flujo moderno: el evento de negocio sale del outbox del sistema actual, se publica en Pub/Sub

y llega a este worker (módulo Cloud Run). El worker **nunca escribe a ciegas**: cada paso decide

contra un **catálogo-whitelist** (`CATALOG_PAYMENT_COMMITTED_V0.1.json` , reglas visibles, sin

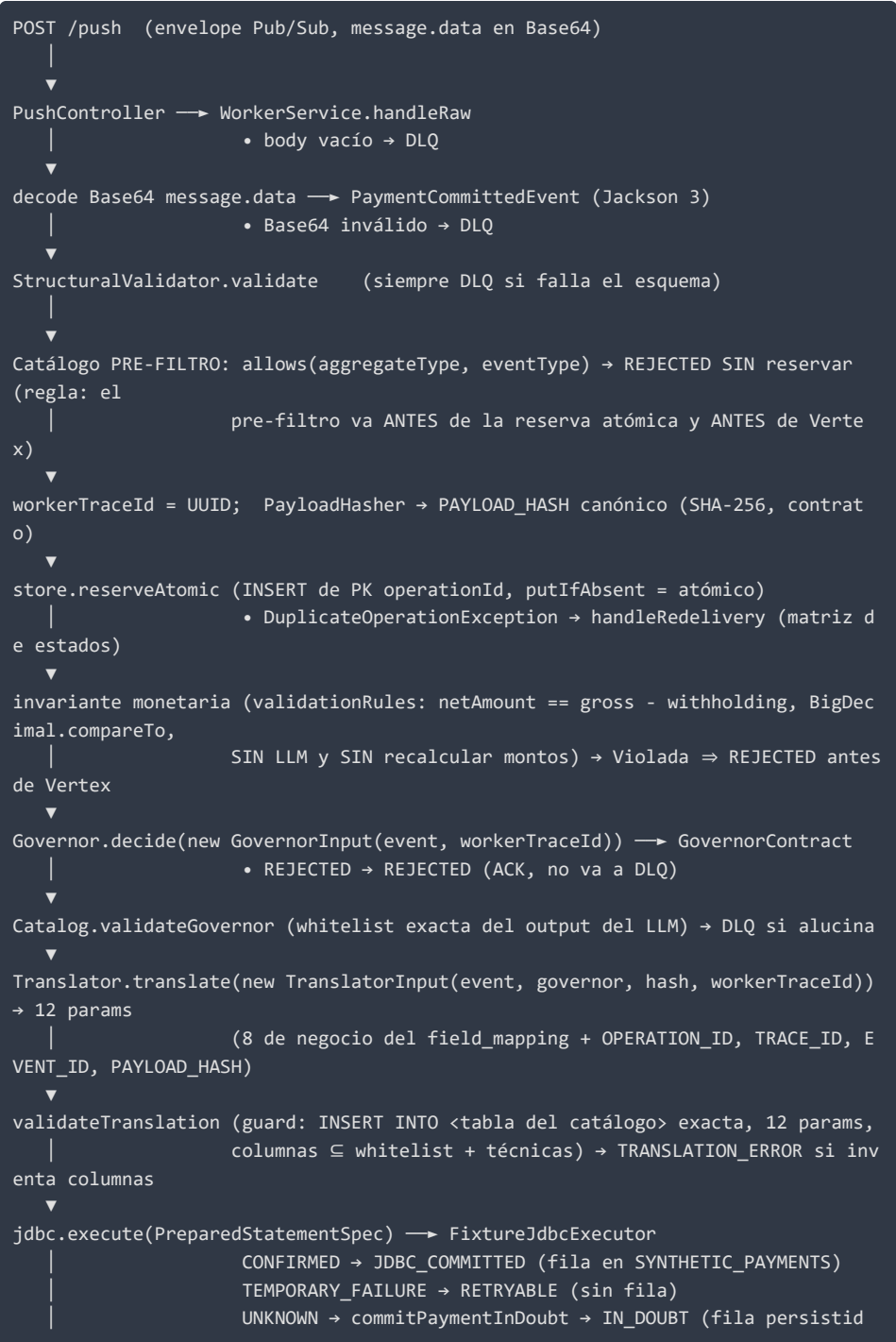
hidden rules) y toda escritura a Sybase es un plan SQL generado por el Traductor con columnas y

parámetros validados campo a campo.

En la PoC los cuatro actores externos están **mockeados de forma determinista**:

Actor externo (prod LNB)	En esta PoC
Pub/Sub (suscripción)	POST HTTP simulado a <code>/push</code> (envelope real)
Gobernador (Vertex AI + LLM)	<code>MockGovernor</code> (aprueba el plan del catálogo; <code>setOverride</code> inyecta alucinación/rechazo)
Traductor	<code>DeterministicTranslator</code> (limitado al catálogo)
Sybase DES	<code>InMemoryStateStore</code> (mock de 2 tablas)
JDBC real	<code>FixtureJdbcExecutor</code> (CONFIRMED / TEMPORARY_FAILURE / UNKNOWN)
Endpoint de reporte	<code>MockResultReporter</code> (SUCCEEDED / CONFLICT 409 / FAILED)

Pipeline canónico (también es el flujo de `WorkerService.handleRaw`):



```

a, resultado incierto)
▼
ResultReporter.report(OperationResult)
|
|      SUCCEEDED → SUCCEEDED + ACK
|      CONFLICT (409) → SUCCEEDED + ACK (ya reportado; NUNCA re-e
ejecuta JDBC)
|
|      FAILED → RETRYABLE + NACK (redelivery reintenta SOLO el re
porte)
▼
ACK (HTTP 200) – solo después de persistir y/o clasificar la evidencia

```

Reglas de ACK/NACK hacia Pub/Sub (contrato de LNB):

Respuesta	HTTP	Significado
ACK	200	El mensaje ya quedó persistido o clasificado (SUCCEEDED, IDEMPOTENT, REJECTED, DLQ...). No se reenvía.
NACK	500	Pub/Sub debe reintentar (RETRYABLE por fallo temporal, evento en vuelo).

Redelivery sin segunda ejecución JDBC (recuperación, regla de oro del plan): si la fila `SYNTHETIC_PAYMENTS` ya existe con el mismo `PAYLOAD_HASH`, la redelivery **nunca re-ejecuta el**

JDBC; según el estado: `JDBC_COMMITTED` / `REPORT_PENDING` / `RETRYABLE` (con fila) → solo reintentar el reporte; `PROCESSING` (con fila, crash post-commit) o `IN_DOUBT` → conciliación a

`SUCCEEDED` por hash; `SUCCEEDED` + mismo hash → `IDEMPOTENT`; mismo `operationId` con **otro** hash

→ DLQ por colisión de PK.

3. Estructura del código: qué hace cada parte

Paquete base: `com.pagaduriasintetica.worker` (modular monolith), separado por **responsabilidad**.

`contract/` — tipos del contrato (records + enums, sin lógica)

Clase	Qué es
<code>PushEnvelope</code> / <code>PushMessage</code>	Envelope HTTP de Pub/Sub (<code>message.data</code> Base64, <code>messageId</code> , <code>publishTime</code> , <code>deliveryAttempt</code>).
<code>PaymentCommittedEvent</code>	Evento de negocio oficial (Alex): <code>eventId</code> , <code>eventType(PAYMENT_COMMITTED)</code> , <code>aggregateType(prizes.payment)</code> , <code>aggregateId</code> , <code>eventVersion(=1)</code> , <code>destinationSystem(SYBASE)</code> , <code>occurredAt</code> , <code>correlationId</code> , <code>operationId</code> , <code>operationData</code> .
<code>OperationData</code>	Datos de negocio (8 campos): <code>paymentId</code> , <code>claimId</code> , <code>status</code> , <code>paymentMethod</code> , <code>grossAmount</code> , <code>withholdingAmount</code> , <code>netAmount</code> , <code>currency</code> (montos <code>BigDecimal</code>).
<code>OperationContext</code>	<code>workerTraceId</code> (generado por el Worker) + <code>correlationId</code> (de Alex); para logs/Vertex/ <code>sync_attempt.trace_id</code> .
<code>GovernorInput</code> / <code>GovernorContract</code>	Entrada al Gobernador (evento, <code>workerTraceId</code>) ; salida (<code>contract_version</code> , <code>decision</code> , <code>operationId</code> , <code>traceId</code> , <code>reason</code> , <code>target_table</code> , <code>required_fields</code> ,

Clase	Qué es
	<code>field_mapping</code> , <code>value_rules</code> , <code>catalog_version</code> , <code>violations</code>).
<code>TranslatorInput</code> / <code>TranslatorResult</code>	Entrada al Traductor (<code>evento</code> , <code>GovernorContract</code> , <code>payloadHash</code> , <code>workerTraceId</code>); salida <code>sql_template</code> + <code>parameters</code> (12) + <code>preview_sql</code> .
<code>PreparedStatementSpec</code>	Plan ejecutable del JDBC: <code>sql_template</code> , <code>target_table</code> , <code>parameters</code> , <code>operationId</code> , <code>payloadHash</code> .
<code>JdbcExecutionResult</code> / <code>JdbcOutcome</code>	Resultado del JDBC: <code>CONFIRMED</code> / <code>TEMPORARY_FAILURE</code> / <code>UNKNOWN</code> .
<code>OperationResult</code> / <code>ReportResult</code> / <code>ReportOutcome</code>	Reporte al endpoint: <code>SUCCEEDED</code> / <code>CONFLICT</code> / <code>FAILED</code> .
<code>ProcessingOutcome</code>	Resultado hacia Pub/Sub: <code>ack</code> + <code>status</code> + <code>operationId</code> + <code>reason</code> .
<code>OperationStatus</code>	<code>PROCESSING</code> / <code>JDBC_COMMITTED</code> / <code>REPORT_PENDING</code> / <code>SUCCEEDED</code> / <code>REJECTED</code> / <code>RETRYABLE</code> / <code>IN_DOUBT</code> / <code>IDEMPOTENT</code> / <code>TRANSLATION_ERROR</code> / <code>DLQ_QUARANTINED</code> .
<code>OperationState</code>	Fila de <code>WORKER_OPERATION_STATE</code> (PK <code>operationId</code> + <code>payloadHash</code> + estado + attempts + decisión). Inmutable (<code>with*</code>).
<code>PaymentRow</code>	Fila de <code>SYNTHETIC_PAYMENTS</code> (<code>operationId</code> + <code>payloadHash</code> + <code>target_table</code> + parámetros).

catalog/

Clase	Qué es
<code>Catalog</code>	Carga <code>catalog/CATALOG_PAYMENT_COMMITTED_V0.1.json</code> del classpath (whitelist). Triple rol: pre-filtro (<code>allows(aggregateType, eventType)</code>), validación exacta del output del Gobernador (<code>validateGovernor</code> , incluida la comparación de <code>catalog_version</code>) y única fuente de tabla/columnas para el Traductor (<code>targetTable</code> , <code>fieldMapping</code> , <code>requiredFields</code> , <code>valueRules</code> , <code>validationRules</code>).

governor/

Clase	Qué es
<code>Governor</code>	Interfaz de decisión (en prod: Vertex AI).
<code>MockGovernor</code>	Default <code>APPROVED</code> con el plan exacto del catálogo; <code>setOverride</code> inyecta comportamientos (alucinar tabla → <code>DLQ</code> , rechazo de negocio → <code>REJECTED</code>).

translator/

Clase	Qué es
<code>Translator</code>	Interfaz: <code>TranslatorInput</code> → <code>TranslatorResult</code> .
<code>DeterministicTranslator</code>	Genera <code>INSERT INTO <target></code> (8 columnas de negocio por <code>field_mapping</code> + <code>OPERATION_ID</code> , <code>TRACE_ID</code> , <code>EVENT_ID</code> , <code>PAYLOAD_HASH</code>) <code>VALUES</code> (?, ×12), aplicando las <code>value_rules</code> autorizadas (status/method/moneda). Solo

Clase	Qué es
	columnas del catálogo: cualquier columna libre = <code>TRANSLATION_ERROR</code> (regla <i>no_invented_columns</i>).

worker/ — orquestador, estado, JDBC y reporte

Clase	Qué es
<code>WorkerService</code>	Orquesta todo el pipeline (ver §2). Decide ACK/NACK y los estados finales. Regla inviolable: el Gobernador nunca se invoca antes de reservar <code>operationId</code> , y la invariante monetaria se evalúa sin LLM.
<code>PushController</code>	<code>POST /push</code> : body crudo → <code>handleRaw</code> , HTTP 200 (ACK) / 500 (NACK).
<code>StructuralValidator</code>	Esquema del evento (no reglas de negocio): <code>eventVersion</code> , IDs, ISO-8601, <code>destinationSystem</code> SYBASE, montos > 0, longitudes.
<code>PayloadHasher</code>	Hash canónico SHA-256 sobre <code>OperationData</code> (<code>TreeMap</code> + <code>BigDecimal.setScale(2)</code>). Constantes <code>CONTRACT_VERSION</code> , <code>EVENT_TYPE</code> , <code>AGGREGATE_TYPE</code> , <code>EVENT_VERSION</code> .
<code>OperationStateMachine</code>	Política de transiciones centralizada; toda mutación pasa por ella; transiciones inválidas (p. ej. <code>SUCCEEDED→PROCESSING</code>) lanzan <code>IllegalStateException</code> .
<code>StateStore</code> / <code>InMemoryStateStore</code>	Interfaz y mock de Sybase: <code>WORKER_OPERATION_STATE</code> + <code>SYNTHETIC_PAYMENTS</code> + auditoría de cuarentena. <code>reserveAtomic</code> usa <code>putIfAbsent</code> (= INSERT de PK; prohibido check-then-act). Helpers de fixture: <code>seedDoubtful</code> , <code>seedInDoubt</code> , <code>seedJdbcCommitted</code> , <code>commitPayment</code> , <code>commitPaymentInDoubt</code> .
<code>DuplicateOperationException</code>	Señala colisión de PK → redelivery por matriz de estados.
<code>JdbcExecutor</code> / <code>FixtureJdbcExecutor</code>	Interfaz del JDBC y ejecutor del fixture (<code>executeFixtureTransaction()</code>): <code>CONFIRMED/TEMPORARY_FAILURE/UNKNOWN</code> ; <code>executeCount()</code> y <code>setForcedOutcome</code> para pruebas de no-reejecución.
<code>ResultReporter</code> / <code>MockResultReporter</code>	Interfaz del endpoint de reporte (Opción B) y su mock: <code>SUCCEEDED/CONFLICT/FAILED</code> con <code>setOverride</code> .

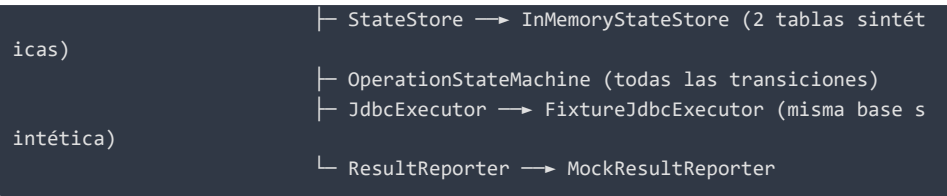
4. Cómo se conectan las piezas

Grafo real de Spring de la prueba vertical (beans reales, contexto completo):

```

WorkerPocApplication
├─ PushController → WorkerService
│   ├── ObjectMapper (Jackson 3, autoconfig de Boot)
│   ├── PayloadHasher
│   ├── StructuralValidator
│   └── Catalog (carga el JSON del classpath al constructor)
├─ Governor → MockGovernor
└─ Translator → DeterministicTranslator

```



Los records del `contract/` son los "envelopes" entre etapas: `PushEnvelope` (HTTP) → `PaymentCommittedEvent` (dominio) → `GovernorContract` (salida del Gobernador) → `TranslatorResult` (SQL) → `PreparedStatementSpec` (JDBC) → `OperationResult` (reporte) → `OperationState` / `PaymentRow` (persistencia). Al cambiar un actor (p. ej. Gobernador por su cliente Vertex AI real) solo hay que dar otra implementación de la interfaz: el resto del pipeline no cambia.

Particularidad técnica: el proyecto usa **Jackson 3** (Spring Boot 4), por eso los imports son `tools.jackson.databind.*` y no `com.fasterxml.jackson.*`.

5. Estados del contrato y transiciones

Estados del contrato (`OperationStatus`):

Estado	Cuándo se emite	ACK/NACK
PROCESSING	Reserva atómica hecha, pipeline en curso	—
JDBC_COMMITTED	<code>executeFixtureTransaction()</code> confirmó el INSERT (fila persistida)	—
REPORT_PENDING	JDBC confirmado, enviando reporte al endpoint	—
SUCCEEDED	Reporte OK (o 409 ya reportado, o in-doubt conciliado por hash)	ACK
IDEMPOTENT	Mismo <code>operationId</code> + mismo <code>PAYLOAD_HASH</code> ya procesado	ACK
RETRYABLE	Fallo JDBC temporal (sin fila), fallo de reporte (con fila) o evento en vuelo	NACK
IN_DOUBT	Resultado JDBC indeterminado (UNKNOWN): commit posiblemente aplicado	NACK (redelivery concilia)
REJECTED	Pre-filtro del catálogo (sin Vertex) o invariante monetaria o rechazo del Gobernador (paso 9). No va a DLQ.	ACK
DLQ_QUARANTINED	Base64/JSON malformados, alucinación del Gobernador, colisión de PK (otro hash), in-doubt con hash inconsistente	ACK
TRANSLATION_ERROR	El Traductor inventa columnas / SQL fuera de whitelist (<code>guard validateTranslation</code>)	ACK

Política de transiciones (`OperationStateMachine`):

```

PROCESSING      → JDBC_COMMITTED, REJECTED, RETRYABLE, IN_DOUBT, TRANSLATION_ERROR, DLQ_QUARANTINED
RETRYABLE       → PROCESSING (reproceso), REPORT_PENDING (solo reporte), DLQ_QUARANTINED
JDBC_COMMITTED  → REPORT_PENDING, SUCCEEDED, RETRYABLE, IN_DOUBT, DLQ_QUARANTINED

```

```

ED
REPORT_PENDING → SUCCEEDED, RETRYABLE, DLQ_QUARANTINED
IN_DOUBT → SUCCEEDED (hash igual), RETRYABLE, DLQ_QUARANTINED (hash dist
into)
SUCCEEDED / REJECTED / TRANSLATION_ERROR / DLQ_QUARANTINED → terminales

```

Ejemplos de inválidas: `SUCCEEDED → PROCESSING`, `PROCESSING → SUCCEEDED` (la recuperación

in-doubt pasa por `IN_DOUBT`, no por el salto directo).

La regla de oro es de LNB: **ACK solo cuando ya hay evidencia persistida o clasificada**; así un

duplicado nunca causa doble pago (mitigado además por el "nunca re-ejecutar JDBC" en redelivery).

6. Pruebas y evidencia

Suite	Qué cubre
<code>WorkerPocApplicationTests</code>	Smoke: el contexto Spring completo arranca.
<code>WorkerPipelineTest</code> (24)	E2E con beans reales: los 6 casos de LNB + subcasos in-doubt + las ~15 pruebas nuevas del plan (Fase 5).
<code>BranchingCoverageTest</code> (4)	Cobertura por rama (Mockito puro, sin ApplicationContext): pre-filtro sin reservar, invariante antes del Gobernador, rechazo del Gobernador, traducción inventada sin JDBC.
<code>OperationStateMachineTest</code> (4)	Política de transiciones: válidas, redelivery desde JDBC_COMMITTED, terminales, salto inválido.
<code>PayloadHasherTest</code> (3)	Hash canónico del contrato, normalización <code>200/150</code> y equivalencia decimal <code>1.10 == 1.1</code> .
<code>DemoEvidenceTest</code>	Harness E2E de la demo : matriz de 16 escenarios contra el pipeline real y exporta la evidencia.
<code>TestEnvelopeFactory</code>	Helpers de test (fixture único + <code>HASH_OP001</code> , no es una suite).
<code>EvidenceWriter</code>	Exporta la evidencia a <code>target/demo/</code> .

Los 16 escenarios del harness: caso1 → SUCCEEDED; caso2a → IDEMPOTENT; caso2b → RETRYABLE (en

vuelo); caso2c → DLQ (PK collision, ACK DLQ sin degradar el SUCCEEDED); caso3 → REJECTED

(pre-filtro del catálogo, sin reservar); caso3b → REJECTED (eventType no permitido);

caso4 → DLQ

(Gobernador alucina tabla); caso5 r1/r2/r3 → in-doubt recuperado / retry reprocesado / in-doubt

con hash distinto (DLQ); caso6 x3 → DLQ (Base64 corrupto, JSON malformado, campos faltantes);

paso9 → REJECTED (decisión del Gobernador); translation → TRANSLATION_ERROR (columna

inventada); hasher → PASS (hash del contrato). Cada escenario registra su expediente y el

`@AfterAll` exporta `evidencia_casos.json`, `EVIDENCIA_POC_LNB.md` y los envelopes reutilizables.

Reproducir la evidencia

```
mvn test
```

Regenera `target/demo/` (gitignored). Snapshot congelado de la entrega en `docs/evidencia/2026-09-22/` (VO sintética histórica en `docs/evidencia/2026-09-15/`).

7. Demo en vivo (HTTP)

```
mvn -DskipTests package
.\scripts\demo-live-start.ps1      # arranca el jar en :8080 y espera health UP
.\scripts\demo-live-run.ps1        # POSTea target\demo\envelopes\live\*.json a
./push
.\scripts\demo-live-stop.ps1       # detiene el worker por el listener de :8080
```

Resultado esperado: 7/7 HTTP 200 (`SUCCEEDED` , `IDEMPOTENT` , `REJECTED` ×2, `DLQ_QUARANTINED` ×3); también se guarda `target/demo/demo-live-run.json` . Más detalle en `DEMO.md` .

8. Requisitos y particularidades del entorno

- **Java 21 y Maven 3.9+** (o el wrapper `mvnw`). Spring Boot 4.1.1 → **Jackson 3** (`tools.jackson`).
 - En Windows PowerShell, los scripts de live usan `Start-Process` y matan el worker por el **listener de :8080** (no por PID), porque el PID registrado puede ser el del shim y no el del JVM.
 - Puerto: `8080` . Health check: `actuator` (`/actuator/health`).
-

9. Limitaciones declaradas de la PoC

- La PoC corre **todo en mocks deterministas**: `MockGovernor` (Vertex AI), `InMemoryStateStore` (Sybase/JDBC), `MockResultReporter` (endpoint HTTP). **No hay Vertex, Pub/Sub real, JDBC ni HTTP al endpoint**. `FixtureJdbcExecutor` y `MockResultReporter` son los puntos de sustitución.
- La **atomicidad local del fixture NO demuestra la atomicidad Sybase-almacenamiento del Worker** en integración real (un UNKNOWN real requiere decidir entre commit y no-commit).
- El escenario "evento en vuelo" es **simulado** (registro PROCESSING pre-sembrado), no concurrencia real de hilos.
- El catálogo es el establecido por Alex (`CATALOG_PAYMENT_COMMITTED_V0.1.json` , `SYNTHETIC_PAYMENTS` placeholder de `prize.payment`); el fixture SQL es DDL de DEV.
- El escenario de hash de contrato es una función pura (sin HTTP). Puntos de sustitución en producción: `Governor` (Vertex AI), `Translator` , `StateStore` (JDBC real vía jConnect) y `ResultReporter` (HTTP al endpoint oficial) — el resto del pipeline

queda
intacto.

10. Estado para integración con DEV (PoC vs real)

Componente	Estado en PoC	Estado en DEV	Responsable	Próxima fecha
Gobernador (Vertex AI)	MockGovernor	SDK + prompt real	Carlos / Steven	9-10 oct (scaffold)
Pub/Sub (push real)	POST HTTP simulado	Suscripción + DLQ real	JD	14-15 oct (config)
JDBC → Sybase	FixtureJdbcExecutor	Driver jConnect + VPN	Alex	Driver pendiente
Reporte a API	MockResultReporter	HTTP real con 409 codes	Steven + Alex	Contrato en revisión
Almacenamiento state	InMemoryStateStore	JDBC durable (idempotencia entre instancias)	Steven	20 oct (integración)
Outbox → Pub/Sub	No existe	Publicador transaccional	Steven (propuesta)	11-15 oct

Invariants del commit (regla de oro del plan)

1. **"Commit significa éxito"**: tras `JDBC_COMMITTED`, la operación **nunca** se reejecuta ni se degrada a DLQ. Un fallo de reporte queda como `REPORT_PENDING` con `reportStatus` y `manualActionRequired`, respondiendo ACK para sacar el mensaje de la suscripción.
2. **ACK solo con evidencia**: `ack=true` (HTTP 200) implica que hay fila persistida o estado clasificado. `ack=false` (HTTP 500) → Pub/Sub reintenta.
3. **DLQ funcional ≠ DLQ técnico**: `DLQ_QUARANTINED` (cuarentena funcional) se ACK y queda en el worker; el DLQ nativo de Pub/Sub recibe solo NACKs técnicos agotados.
4. **UNKNOWN nunca se reejecuta**: → `IN_DOUBT`, conciliación posterior por `PAYLOAD_HASH`.

11. Próximas tareas (roadmap E3.x)

Tarea	Responsable	Estado	Fecha objetivo
E3.0 — Estado base	Steven	✅ Completado (15e2368)	—
E3.1 — Gobernador real (Vertex)	Carlos	⌚ Pendiente SDK/prompt	16/10 (review)
E3.2 — Pub/Sub real + 16 payloads	Steven	⌚ Pendiente suscripción JD	19/10 (ensayo)
E3.3 — JDBC real → Sybase	Alex + Steven	⌚ Pendiente driver+VPN	20/10 (integración)
E3.4 — Almacenamiento durable	Steven	⌚ Pendiente JDBC real	20/10
E3.5 — Reporte HTTP + 409 codes	Steven + Alex	⌚ Pendiente contrato	16/10

Tarea	Responsable	Estado	Fecha objetivo
E3.6 — Coordinación/contratos	Todo el equipo	 Pendiente JD/Alex	20/10

Checklist Go/No-Go 16 octubre

- ☐ SDK Vertex en `pom.xml` con prompt funcional (E3.1)
- ☐ Suscripción push + `--invoker` configurado por JD
- ☐ Driver jConnect recibido de Alex (procedencia/licencia/checksum)
- ☐ Credenciales Secret Manager operativas
- ☐ Conectividad JDBC desde Cloud Run → Sybase (`192.168.2.14:5000`)
- ☐ Contrato 409 del reporte cerrado con Alex

12. Referencias

- `lnb-docs/CONTRACT_EVENTO_V0.1.md` — contrato oficial PAYMENT_COMMITTED de Alex (base).
- `lnb-docs/PLAN_IMPLEMENTACION_V0.1.md` — plan CONGELADO que ejecuta esta implementación.
- `lnb-docs/MINUTA_ALEX_RESULTADO_V1.md` — minuta/decisiones de Alex.
- `lnb-docs/AMBIENTE_DEV_LNB.md` — ambiente y seguridades del DEV.
- `lnb-docs/PRUEBA_VERTICAL_CASOS_SYNTHETIC_V0.md` — casos originales que reproduce el harness.
- `lnb-docs/PLAN_POC_JAVA_V1.md` — plan de la PoC Java (fases, decisiones, anexos).
- `lnb-docs/REPORTE_EVIDENCIA_POC_LNB.md` — reporte con índices a toda la evidencia.
- `src/main/resources/catalog/CATALOG_PAYMENT_COMMITTED_V0.1.json` — whitelist real del worker.
- `docs/evidencia/2026-09-22/README.md` — cómo leer el snapshot congelado de la entrega.
- `DEMO.md` — runbook corto de la demo.

13. Requisitos de despliegue (Docker / Cloud Run)

Ítem	Valor
Dockerfile	Multi-stage: build JDK 21.0.6_7 → runtime JRE 21.0.6_7, digest-fijado
Usuario runtime	<code>uid=1001(lnb)</code> (no root)
Puerto	<code>8080</code> (inyectado por Cloud Run vía <code>PORT</code>)
Health check	<code>/actuator/health/readiness</code> → <code>{"status":"UP"}</code>
Imagen	<code>worker-poc:dev</code> (448 MB, local) / Artifact Registry en DEV
Variables	Ver <code>.env.example</code> — todo inyecta por <code>APP_*</code>
Service account	<code>sa-run-agentes-dev-lnb@proy-comercial-dev-lnb.iam.gserviceaccount.com</code>
Perfil DEV	<code>dev-vertex</code> para Gobernador real (<code>--spring.profiles.active=dev-vertex</code>)
Permiso push	<code>run.invoker</code> sobre el servicio (JD lo habilita)

Hash canónico: 0fd5240accd7b41a9d55d95567eb79b0f87a9776c0396a5f888d9c7a99c246a8